EXERCICE RÉCAPITULATIF — GIT & REACT

Tu développes une application React appelée TaskFlow, qui permet de gérer une liste de tâches. Le dépôt GitHub existe déjà. Tu dois récupérer le projet, développer des fonctionnalités, collaborer avec un collègue, gérer les branches et corriger des erreurs.

A. Récupérer et vérifier le projet

1. Télécharger le dépôt GitHub sur ton ordinateur.
2. Afficher les dépôts distants configurés.
3. Afficher les URL des dépôts distants.
4. Afficher l'état des fichiers du projet.
5. Récupérer et intégrer les modifications de ton collègue.
6. Récupérer les informations du dépôt distant sans intégrer les modifications.
7. Afficher l'historique des commits sous forme courte.
8. Afficher les différences entre les fichiers modifiés et le dernier commit.

B. Ajouter et sauvegarder les modifications

9. Ajouter uniquement src/App.jsx à la zone de staging.
10. Ajouter toutes les modifications du projet à la zone de staging, y compris les suppressions.
11. Créer un commit avec le message Ajout de la recherche.
12. Envoyer les commits locaux sur GitHub.
13. Envoyer une nouvelle branche sur GitHub et configurer son suivi distant.
14. Afficher les détails du dernier commit.

C. Supprimer et annuler des modifications

15. Supprimer debug.log du disque et préparer sa suppression dans Git.
16. Arrêter de suivre debug.log tout en conservant le fichier sur le disque.
17. Annuler les modifications locales de src/App.jsx sans toucher aux autres fichiers.
18. Retirer src/App.jsx du staging tout en conservant ses modifications.
19. Annuler toutes les modifications non commitées des fichiers suivis et vider le staging.

20. Annuler le dernier commit en conservant les modifications dans le staging.
21. Annuler le dernier commit en conservant les modifications dans les fichiers, sans staging.
22. Annuler le dernier commit en conservant les modifications dans le staging, puis vérifier l'historique.
23. Annuler un commit déjà partagé sur GitHub sans supprimer l'historique existant.
24. Restaurer src/App.jsx depuis un commit précis de l'historique.

D. Créer et gérer les branches

25. Afficher les branches locales.
26. Afficher les branches locales et distantes.
27. Créer une branche feature/search sans changer de branche.
28. Basculer vers feature/search avec git switch.
29. Créer et rejoindre feature/header avec une seule commande git checkout.
30. Créer et rejoindre feature/profile avec une seule commande git switch.
31. Revenir sur la branche main avec git checkout, puis avec git switch.
32. Renommer la branche courante en feature/tasks.
33. Supprimer la branche locale feature/header.
34. Fusionner feature/search dans main.
35. Supprimer une branche distante devenue inutile.

E. Collaborer et synchroniser le projet

36. Afficher les URL du dépôt distant.
37. Ajouter un dépôt distant nommé upstream.
38. Modifier l'URL du dépôt distant origin.
39. Récupérer les modifications distantes sans les fusionner.
40. Intégrer les dernières modifications de main dans la branche courante.
41. Envoyer la branche feature/search sur GitHub.
42. Récupérer une branche distante et créer sa branche locale de suivi.

F. Situations pratiques

43. Tu as modifié trois fichiers React, dont un nouveau composant SearchBar.jsx. Ajouter toutes les modifications, créer un commit Ajout du composant SearchBar, puis envoyer les changements sur GitHub.
44. Tu as commité une modification incorrecte sur main, mais tu n'as pas encore envoyé ce commit. Revenir au commit précédent tout en conservant les modifications dans les fichiers.
45. Ton collègue a envoyé un commit incorrect sur GitHub. Annuler ses effets avec un nouveau commit, sans réécrire l'historique partagé.
46. Tu travailles sur feature/search. Récupérer les dernières modifications de main, fusionner main dans ta branche, résoudre les conflits éventuels, puis envoyer la branche sur GitHub.
47. Tu as terminé feature/search. Basculer sur main, récupérer les dernières modifications distantes, fusionner feature/search, puis envoyer main sur GitHub.
48. Une erreur a été commise pendant la fusion de deux branches. Afficher l'état du dépôt et l'historique pour identifier la situation avant de corriger le problème.

Correction :

Correction complète — Exercice Git & React

Voici les commandes attendues pour les 48 questions, dans le même ordre que l'exercice. Les commandes supposent que les branches et les dépôts mentionnés existent, sauf lorsqu'il est demandé de les créer.

A. Récupérer et vérifier le projet

1 git clone <URL_DU_DEPOT>

2 git remote

3 git remote -v

4 git status

5 git pull

6 git fetch

7 git log --oneline

8 git diff

B. Ajouter et sauvegarder les modifications

Nº Correction

9 git add src/App.jsx

10 git add -A

11 git commit -m "Ajout de la recherche"

12 git push

13 git push -u origin <nom-branche>

14 git show HEAD

C. Supprimer et annuler des modifications

Nº Correction

15 git rm debug.log

16 git rm --cached debug.log

17 git restore src/App.jsx

18 git restore --staged src/App.jsx

19 git reset --hard HEAD

20 git reset --soft HEAD~1

21 git reset --mixed HEAD~1

22 git reset --soft HEAD~1 puis git log --oneline

23 git revert <hash-du-commit>

24 git restore --source=<hash-du-commit> -- src/App.jsx

À retenir : les trois commandes reset se distinguent par ce qu'elles conservent après le déplacement de HEAD.

- --soft : conserve les modifications dans le staging.
- --mixed : conserve les modifications dans les fichiers, mais les retire du staging.
- --hard : remet les fichiers suivis et le staging à l'état du commit ciblé, en abandonnant les modifications concernées.

D. Créer et gérer les branches

Nº	Correction
25	git branch
26	git branch -a
27	git branch feature/search
28	git switch feature/search
29	git checkout -b feature/header
30	git switch -c feature/profile
31	git checkout main puis git switch main
32	git branch -m feature/tasks
33	git branch -d feature/header
34	git switch main puis git merge feature/search
35	git push origin --delete <nom-branche>

E. Collaborer et synchroniser le projet

Nº	Correction
36	git remote -v
37	git remote add upstream <URL_DU_DEPOT>
38	git remote set-url origin <NOUVELLE_URL>

39 git fetch origin

40 git merge origin/main

41 git push -u origin feature/search

42 git fetch origin puis git switch --track origin/<nom-branche>

F. Situations pratiques

43. Ajouter tous les fichiers, créer un commit et envoyer les modifications.

```
git add -A
git commit -m "Ajout du composant SearchBar"
git push
```

44.Annuler le dernier commit local tout en conservant les modifications dans les fichiers.

```
git reset --mixed HEAD~1
```

45. Annuler un commit déjà partagé sans réécrire l'historique.

```
git revert <hash-du-commit>
git push
```

46. Récupérer main, fusionner dans feature/search, puis envoyer la branche.

```
git switch feature/search
git fetch origin
git merge origin/main
# Résoudre les conflits éventuels, puis :
git add -A
git commit -m "Résolution des conflits"
git push
```

Si aucun conflit ne survient, il n'est pas nécessaire de créer le commit de résolution.

47. Fusionner feature/search dans main et envoyer main sur GitHub.

```
git switch main
git pull
git merge feature/search
git push
```

48. Examiner le dépôt après une fusion problématique.

```
git status
git log --oneline --graph --all
git diff
```

Les commandes essentielles à mémoriser

Commande	**Rôle**
git add -A	Ajouter toutes les modifications au staging
git commit -m "message"	Créer un commit
git pull	Récupérer et intégrer les modifications distantes
git push	Envoyer les commits vers le dépôt distant
git fetch	Récupérer les informations distantes sans les intégrer
git rm fichier	Supprimer un fichier et préparer sa suppression
git reset --soft HEAD~1	Annuler le dernier commit en conservant le staging
git reset --mixed HEAD~1	Annuler le dernier commit en conservant les modifications hors staging
git reset --hard HEAD	Abandonner les modifications locales suivies
git revert <hash>	Annuler un commit par un nouveau commit
git restore fichier	Abandonner les modifications locales du fichier
git restore --staged fichier	Retirer le fichier du staging
git branch	Afficher les branches locales
git switch -c nom	Créer et rejoindre une branche
git checkout -b nom	Créer et rejoindre une branche avec l'ancienne syntaxe
git merge nom	Fusionner une branche
git log --oneline	Afficher l'historique simplifié
git remote -v	Afficher les URL des dépôts distants